Module 6

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 6 Study Guide and Deliverables

February 20 – February 26

Theme:	Cyber System Security
Topics:	• Hardware security: meltdown, spectre, TEE • Virtualization and Cloud computing security • Mobile security and IoT security
Readings:	• Chapter 8 and Chapter 13.1
Assignments:	• Assignment 3 due **Tuesday, February 27 at 6:00 AM ET**
Assessments:	• Quiz 6 due **Tuesday, February 27 at 6:00 AM ET**
Discussions:	• None
Live Classrooms:	• Lecture: **Tuesday, February 20, 7:30-9:00 PM ET** • Final Exam Review: **Saturday, February 24 8:00-9:00 PM ET** • Facilitator office hour: TBD
Course Evaluation:	Course Evaluation opens on **Monday, February 26, at 10:00 AM ET** and closes on **Sunday, March 3, at 11:59 PM ET**. Please complete the course evaluation. Your feedback is important to MET, as it helps us make improvements to the program and the course for future students.

Topic 1: Virtualization

Learning Outcomes

By the end of this module, you will be able to:

- Describe the fundamentals of virtualization, cloud computing, and mobile and IoT systems.
- Describe major security threats in cloud computing, smartphones, and IoT systems.
- Describe how virtualization can help improve the security.
- Describe the general permission model used in smartphone systems and its limitations.

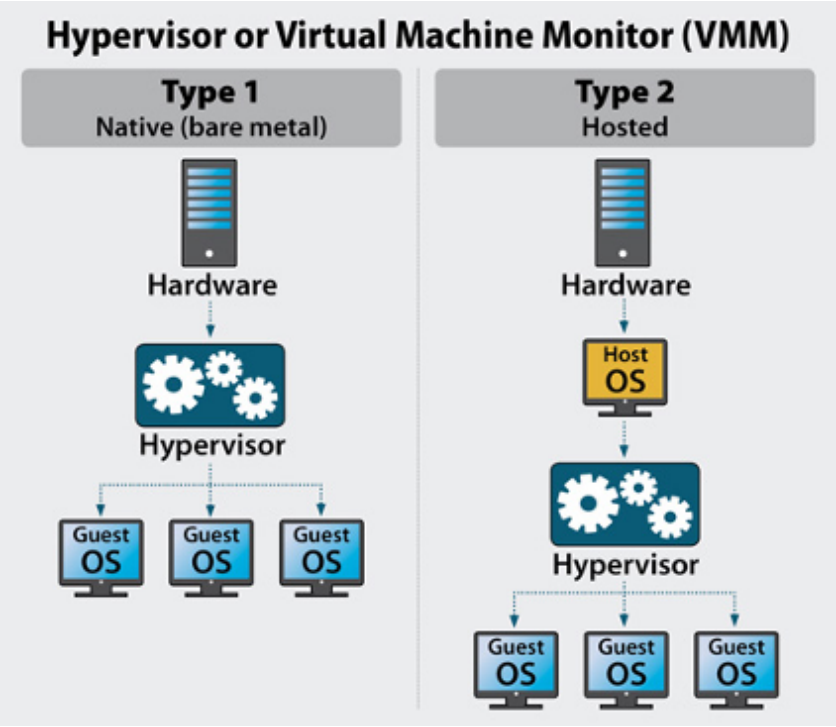
Introduction

In this module, we will provide an overview of various security branches, including virtualization security, cloud computing security, mobile security and IoT security.

Virtualization Basics

Virtualization presents the operating system (OS) with the illusion that it is directly running on a physical platform by emulating the required hardware, e.g., CPU, memory, hard drive, and I/O devices. This type of emulated platform is called a **virtual machine** (VM) (in contrast to a physical machine), and the OS running inside the virtual machine is typically called the **Guest OS**. The software that emulates the virtual platform is called the **virtual machine monitor** (VMM) or **hypervisor**. The OS where the hypervisor is running is usually named the **Host OS**. The figure below illustrates that a user is running Mac OS as the Host OS, VMware as the hypervisor, and Kali Linux as the Guest OS. With sufficient physical hardware resources, the user can start multiple VMs, and run different Guest OSes on each VM. Some hypervisors also allow the Guest OS and Host OS to run different instruction sets, e.g., the Host OS runs the Intel X86 instruction set and the Guest OS runs the ARM instruction set. Probably, you have seen an Android system (Guest OS) run in an emulator (VM) on top of Windows OS (Host OS) on Intel processors. This is due to the dynamic binary translation performed by the hypervisor, which translates the ARM instructions into Intel instructions.

We have two types of hypervisors, **Type 1 hypervisor** and **Type 2 hypervisor**, which differ based on whether they run directly on top of physical hardware. A Type 1 hypervisor, also called a native or bare metal hypervisor, runs directly on top of the hardware (without a Host OS), and provides emulated hardware for Guest OSes, VMware ESX servers, etc. In contrast, a Type 2 hypervisor runs as a software on a Host OS, e.g., VMware workstation or VMware Fusion, VirtualBox, etc. As you may notice, a Type 1 hypervisor should be more efficient than a Type 2 hypervisor, since there is no extra Host OS layer in the middle. A Type 1 hypervisor is not as user-friendly as a Type 2, so it is mainly used in commercial environments.



(from Hypervisor Server)

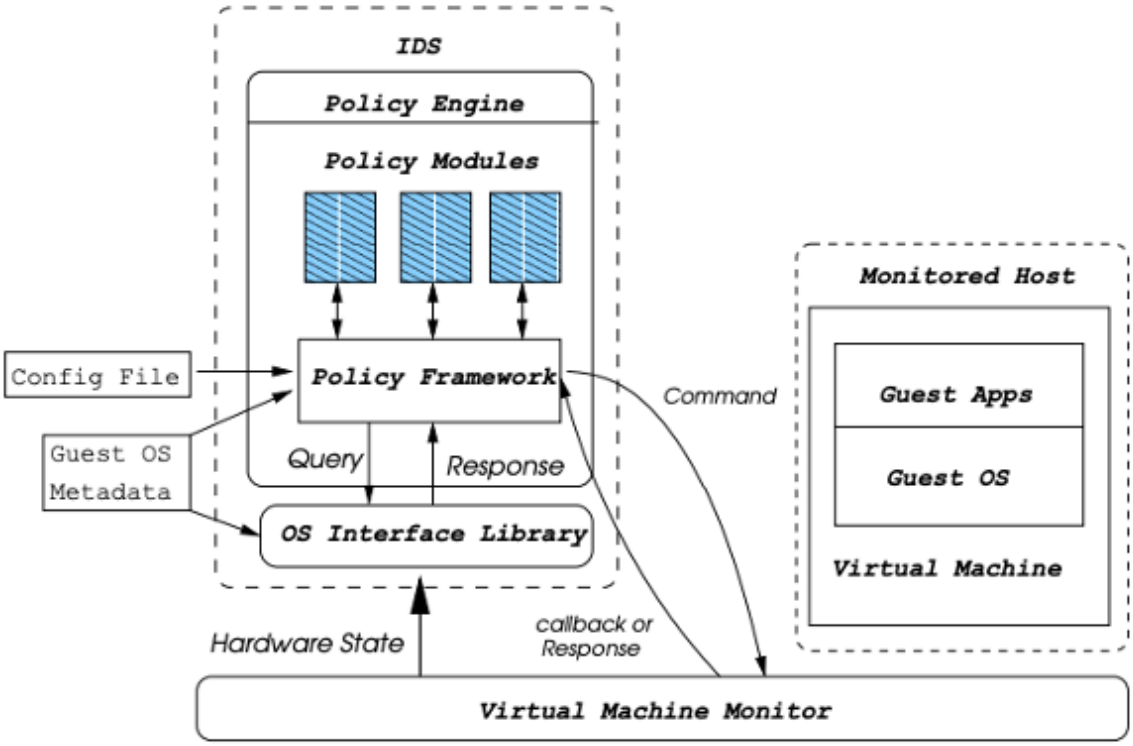
How Virtualization Helps Security

Isolation

Multiple VM instances can run on the same physical platform, and the hypervisor typically isolates each VM instance by allocating dedicated memory space for them to ensure minimal interference. It is as though each VM instance is running on a dedicated machine, using different CPU, memory, and hard drive to run. One can start two VMs, with one used as a trusted environment to run office tasks and financial transactions, and the other as a regular environment to run entertainment applications. Due to the isolation at the lower virtualization layer, even if the regular VM is compromised (e.g., by downloading a third-party game, which happens to be malware), the trusted VM can still maintain its integrity to run the office tasks and financial transactions dependably. Without virtualization, users have to run those trusted services together with the entertainment applications inside the same OS, which exposes the former to risk. An alternative is to purchase two computers and run these different services in different computers, which achieves security but can be quite costly.

VM Introspection

VM introspection was first proposed by Garfinkel and Rosenblum (Garfinkel, 2003) to monitor the execution of a VM in real time, which is quite important for VM-based forensics and intrusion analysis. A fundamental problem of intrusion detection or analysis tools is how to isolate and protect them from being corrupted by attacks/malware. The VM introspection approach utilizes the isolation feature of virtualization; that is, it runs the tools outside of the system it protects. The figure below shows the monitored host (i.e., the OS that we aim to protect) is running in a VM, while the intrusion detection system (IDS) is running outside of it, facilitated by the hypervisor (or VMM). The IDS can either send commands directly to the monitored host for the current state, or read hardware information (including CPU, memory, hard drive, etc.) used by the monitored host from the hypervisor. IDS can analyze the state information of the monitored host against the pre-deployed policy to detect abnormalities. Using the traditional approach that runs the IDS inside the monitored host, the IDS policy and functionality may also be corrupted by attackers when the monitored host is fully controlled by attackers. This is not possible for VM introspection-based IDS, since it is outside of the monitored host, and isolated from it by virtualization layer, thus being able to detect advanced attacks against the monitored host.



VM Introspection based Intrusion Detection System (from Garfinkel [2003])

Honeypot

A honeypot is a controlled environment intended to lure attackers. Usually, the honeypot intends to expose some vulnerabilities and disclose some (faked) sensitive information, thus appearing attractive to the attackers. Meanwhile, the attackers' activities are closely monitored by administrators or security researchers to learn more about their objectives, tools, techniques, and weaknesses. Virtualization is an ideal technique for deploying a honeypot due to the controlled environment offered by isolation. For example, one VM instance can be intentionally configured to attract attackers by using poor protection, leaving lots of ports open, running unpatched software/services, etc. Such an instance is isolated from other VM instances and the Host OS, and can even be deployed in a dedicated local subnetwork. Hence, even if it is compromised and fully controlled by attackers, the damage can be easily contained. Furthermore, VM introspection tools can be used to examine the activities of attackers.

Forensics

Modern hypervisors provide lots of useful functionality besides virtualization, e.g., snapshot, record and play, etc. Snapshot is a checkpoint of the state of the virtual machine, including data in memory areas and the hard drive, execution state of CPU, etc. Users can restore to a previously saved snapshot and continue using the VM from there if anything is wrong with their current VM (e.g., corruption or a compromise). Record and replay extend the functionality of snapshot. Users not only can restore to a previous saved state, but also "replay" all the operations

that have been done since that saved state to “catch up” to the current state. During replay, users have the opportunity to review what has happened on their VM since the last checkpoint, which is quite useful in the field of forensics or intrusion analysis. For instance, if users find the VM is compromised by attackers, they can choose to replay from a known, healthy (before the attack happened) state, and replay the whole VM execution from there. During replay, users can closely monitor and examine all the operations performed, including how the attackers broke through and what they did, thus pinpointing the attack.

Counterattacks against Virtualization

We learned that virtualization can be used by security researchers or system administrators for various security benefits. However, attackers can also leverage it to launch advanced attacks. Samuel, et al. identified a more advanced attack, called Subvirt, that allows attackers to install a hypervisor below the victim system, thus fully controlling it (King, 2006). Furthermore, hypervisors have been found to contain vulnerabilities as well. Such vulnerabilities can be exploited by attackers from the Guest OS, thus escaping the isolation in order to control the entire virtualization platform. As discussed above, a VM can be utilized to set up a honeypot and run VM introspection. After realizing their malware could be analyzed in a VM, attackers designed so-called split-personality malware. Such malware is able to detect if it runs in an emulated platform (i.e., a VM), and once confirmed, it will just behave legitimately without demonstrating any malicious behavior. However, its true intent will appear once it is not running in a VM anymore.

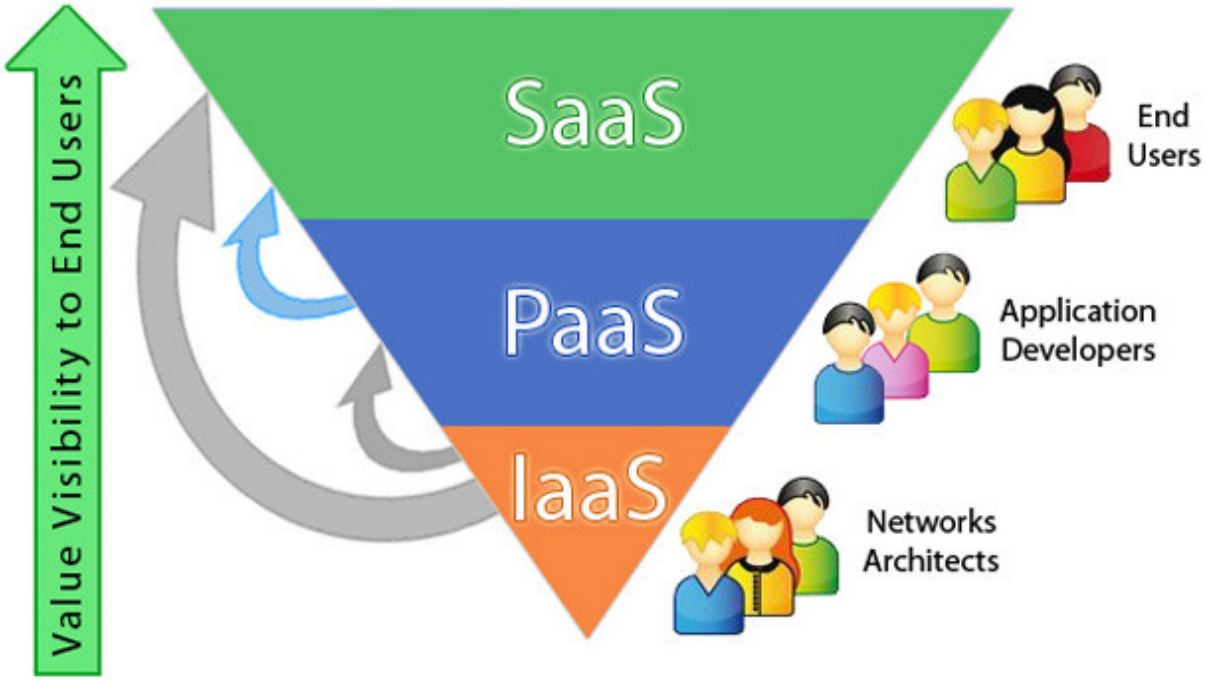
Topic 2: Cloud Computing Security

What Is Cloud Computing?

The National Institute of Standards and Technology (NIST) defines cloud computing as a model “for enabling convenient, on-demand network access to a shared pool of configurable computing resource.” The idea of cloud computing came from organizations and enterprises that owned large data centers with quite low utilization. To utilize their extra computing power, they integrated virtualization technology to deploy VMs on their servers and sold/rented them for revenue. Today, cloud computing involves not only data outsourcing, but also computation outsourcing. The former allows customers to store their data on cloud platforms (e.g., Google Drive, Dropbox, etc.), and the latter provides computation renting services to customers (e.g., Amazon EC2, Microsoft Azure, etc.). Cloud computing features include infinite and elastic resource scalability, on demand, just-in-time provisioning, and no upfront cost (pay-as-you-go).

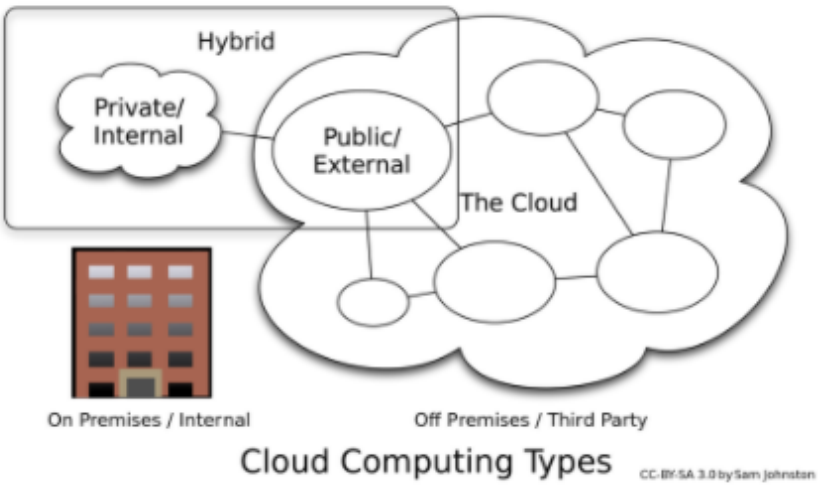
Service Models

There are three basic service models in cloud computing. The first, **Software as a Service (SaaS)**, provides end users with access to the applications running in the cloud (e.g., Google Docs, Salesforce, etc.). End users do not have control over the underlying infrastructure or platform that runs the applications. The second, **Platform as a Service (PaaS)**, offers a development environment to application developers, including an OS, program execution environment, database, and web servers. For instance, Microsoft Azure and Google App Engine allow developers to develop and run their software on the cloud, without requiring the support from the underlying hardware and infrastructure. Finally, **Infrastructure as a Service (IaaS)** allows customers to install any OS and applications on their own, e.g., Amazon EC2, Rackspace Cloud Servers, OpenStack, etc.



Deployment Models

Cloud computing can typically be deployed in three ways: private, public, and hybrid. A private cloud is operated exclusively for a single organization, but its management may be performed internally or contracted out to a third party. A public cloud is available to the general public, either as a paid service or free of charge. A hybrid cloud is composed of both a public cloud and a private cloud. For instance, an organization may store confidential data in a private cloud, but deploy a web service open to the general public on a public cloud.



Cloud Computing

Security Threats in Cloud Computing

Confidentiality

Since customers outsource their data or computation on cloud platforms, the cloud provider needs to be honest, e.g., by running the computation correctly, not manipulating the stored data or computation, etc. Some cloud providers might be honest but curious; that is, they may faithfully provide the required service but peek into the data. Therefore, any outsourced, sensitive data loses confidentiality. Even if the cloud provider is fully trustworthy, customers may still be concerned about loss of control over the outsourced data, e.g., a compromised cloud allows the attackers to access confidential data as well. Furthermore, since a cloud platform stores a large amount of various data from lots of clients, it may run data mining algorithms to analyze the data and learn other aspects about their clients or simply sell client data to third-party data analytics companies if such behavior is not explicitly forbidden in the service contract. Finally, the outsourced data also suffers from **insider threat**. Customers not only need to trust cloud providers, but also have to trust all the employees and contractors hired by the cloud providers. Anyone hired by the cloud providers who has the chance to access the data center should be fully trustworthy as well. They should not intend to steal customers' data or incidentally expose it to the public.

Integrity

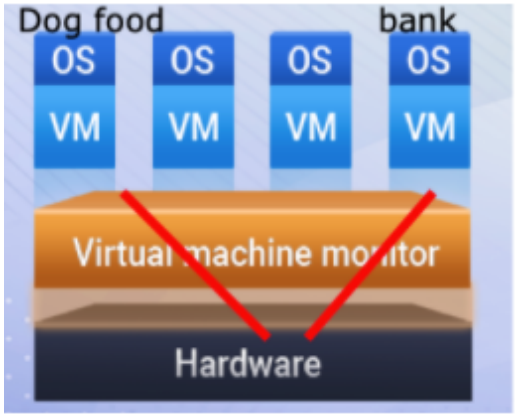
Customers also desire that their outsourced data and computation tasks be correct. Without a local copy of the original data or the capability to run a task locally, it is hard for customers to even know if an integrity compromise has occurred. For instance, a cloud provider may incidentally delete some of the data records on the cloud platform. To save its reputation and to avoid any associated compensation, the cloud provider may just keep quiet without notifying its customers, and may even fake some data records to make up and maintain the overall size of the data records. If customers have a large amount of data stored on the cloud, but do not maintain a good record of what kind of data has been outsourced, they may not be aware of such incidents. Similarly, the correctness of any computation task is hard to track.

Availability

Customers can almost always have their data accessible if they store their data in local data centers. Even in the worst-case scenario of being hacked, they may disconnect their data center from the network, and provide local access only to the data. This is not possible for outsourced data on cloud platforms. If the cloud shuts down due to any Denial of Service (DoS) attack, or the cloud platform needs to be down to clean up damages caused by an intrusion, customers completely lose access to their data. Even worse, what if the cloud provider goes out of business suddenly without early notice? Probably the only solution is to maintain a local copy of the data records as well. Similarly, outsourced web services running on cloud platforms suffer from the same set of problems as data.

Co-Host Threat

As mentioned above, virtualization is the cornerstone of cloud computing. Cloud providers run multiple VM instances on one physical platform and rent each VM instance to their customers to maximize their server resource utilization. For the public cloud however, there is no guarantee that all the VM instances on one physical platform will always be allocated to the same organization. This is due to dynamic provisioning offered by cloud providers. When customers need fewer VM instances, some of their instances on one physical platform may



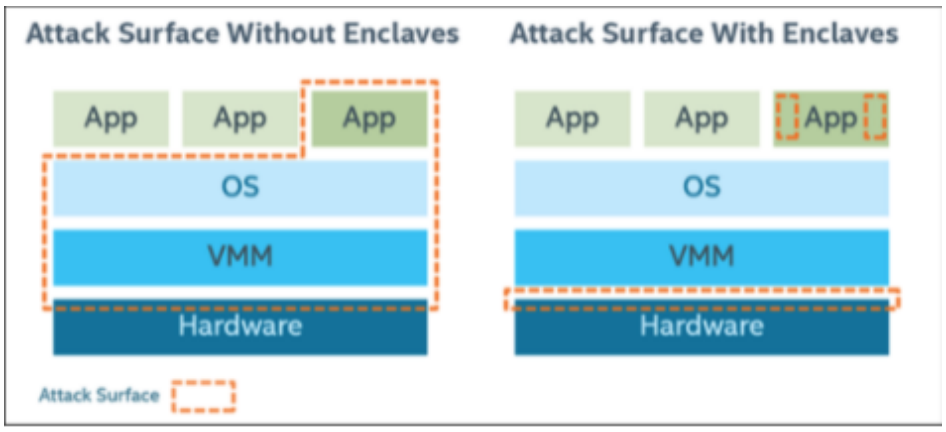
retire and be rented to other organizations, again to maximize the server resource utilization. An intuitive example, shown in the figure here, illustrates a dog food company and a bank with their VM instances running on the same physical platform, sharing CPU, cache, physical memory, hard drive, etc. Note that the isolation of different VM instances we mentioned above is at the logical level, i.e., each VM has its own virtual hardware resources emulated by the hypervisor, but they still share one single set of physical devices. A VM instance could launch attacks against other VM instances running on the same physical platform, which is called co-host threat. Researchers discovered side-channel attacks that leverage shared physical resources, e.g., CPU cache, physical memory, etc., to learn confidential information about one VM instance from another VM instance. (For more details, refer to the white paper Introduction to Side Channel Attacks.) In our example, the security protection of the dog food company might not be comprehensive, and may be compromised by attackers easily. Attackers could then launch a side-channel attack against the VM instance rented to the bank to learn the keys used in the encryption/decryption algorithms of the bank.

Simple Solutions

To ensure data confidentiality and integrity, we learned to use encryption. Customers can encrypt all their data before outsourcing to cloud platforms, so the cloud platforms only observe the ciphertext and cannot decrypt without knowing the key. After downloading data from the cloud, customers need to decrypt the ciphertext to obtain the original data. Certainly, as you may have noticed, the overhead for such encryption and decryption can be significant, especially when customers have a large amount of outsourced data on cloud. Meanwhile, the cloud platforms need to support search over the encrypted data. For instance, customers may request some data based on a keyword, and the cloud provider should be able to use the keyword to search the ciphertext, which needs dedicated algorithms.

Another solution is to leverage a hardware-supported trusted execution environment (TEE), e.g., Intel Software Guard Extensions (SGX) or and AMD Secure Encrypted Virtualization (SEV). TTEE is an isolated and secure runtime space enabled by processor and dedicated memory. Code and data running inside TEE can be protected from VMM and OS with respect to confidentiality and integrity. For instance, beginning with its sixth processors in

2015, Intel integrated SGX support to implement a TEE, called enclave, and reduce the attack surface for trusted application code. In particular, the binary code of one application can be divided into two parts: trusted code and untrusted code. The former can be deployed and run inside the enclave, and the latter runs outside the enclave. As illustrated in the figure below, without enclave (the left diagram), the right app may suffer from attacks from the hardware interface, VMM, OS, and itself. With the trusted code running inside the enclave however, the attack surface is significantly reduced to the hardware interface and the untrusted code only. On cloud platforms with Intel SGX support, customers can run their trusted execution inside an enclave, which is isolated from the underlying VMM and OS managed by cloud providers. A malicious cloud provider cannot compromise the execution inside an enclave from the underlying VMM or OS. Hence, customers now only need to trust Intel processors; it is not necessary to trust cloud providers any more. However, such a solution is still not ideal. Researchers recently found that a side-channel attack is still possible against the code running inside an enclave.



Attack-surface areas with and without Intel Software Guard Extensions enclaves

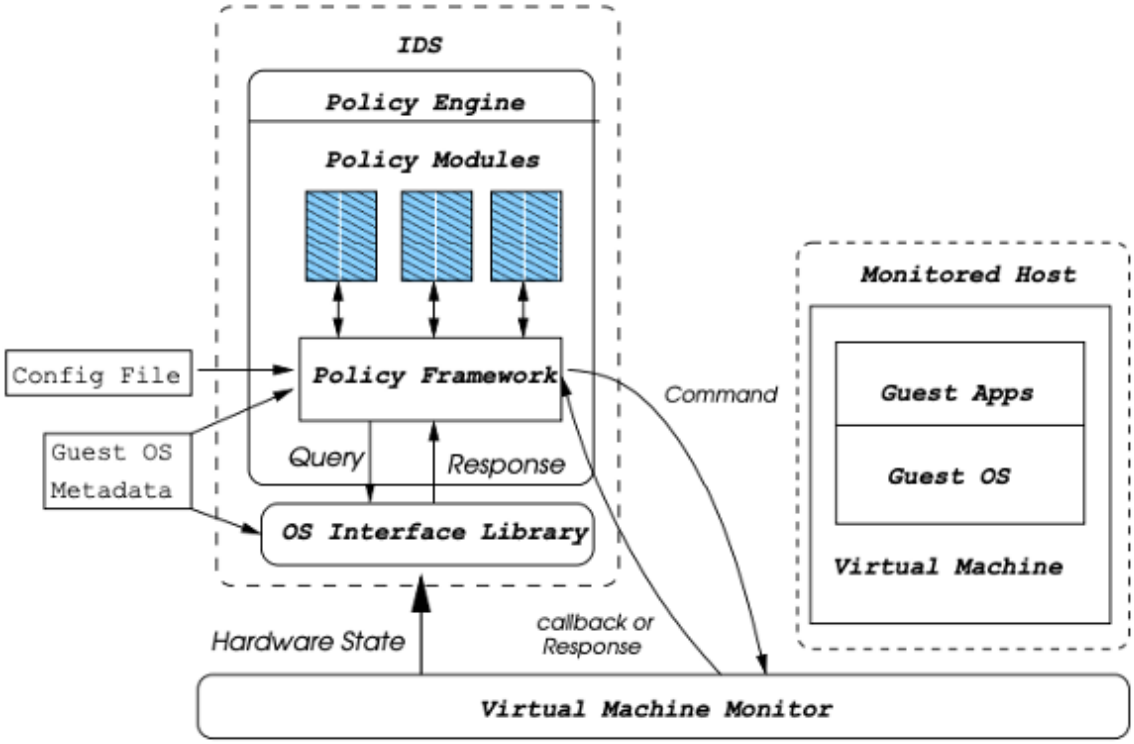
Test Yourself

For the setup of our labs, i.e., VMware, Kali Linux, Metasploitable Linux 2, and the OS you are using on your computer, which is the Guest OS, which is the Host OS, and which is the hypervisor? Did you use a Type 1 or Type 2 hypervisor?

VMware is the hypervisor, and it is Type 2. Kali Linux and Metasploitable Linux 2 are Guest OSes. The answer for Host OS may vary.

Test Yourself

What should be the TCB (trusted computing base) of the IDS in the figure of VM introspection?



Virtual Machine Monitor (hypervisor) and OS interface and library.

Test Yourself

Briefly describe the benefits of using a VM to set up a honeypot.

Refer to the Honeypot section of Virtualization Security in the lecture notes.

Test Yourself

Cloud computing essentially refers to outsourcing data to cloud platforms.

True

False

Test Yourself

Google Docs is an example of SaaS.

True

False

Test Yourself

Amazon EC2 is an example of IaaS.

True

False

Test Yourself

Describe an honest but curious cloud.

Refer to the Confidentiality section of Cloud Computing Security in the lecture notes.

Test Yourself

What is a co-host threat?

Refer to the Co-Host Threat Section of Cloud Computing Security in the lecture notes.

Test Yourself

What is the TCB when using Intel SGX as in the left-hand image of the figure on this page?

Hardware

Test Yourself

Do you think encryption is a good solution to solve confidentiality and integrity issues in the cloud?

Refer to the Simple Solutions Section of Cloud Computing Security in the lecture notes.

Topic 3: Mobile Security

History of Smartphone Systems

Smartphones are very popular; according to a report from Statista, there are 3.5 billion smartphone users all over the world as of 2020. The history of smartphones is not that long. About 20 years ago, Java Platform Micro Edition (Java ME) was the dominant system running on “feature phones.” Such phones could only run a limited number of pre-installed apps with cellular connection only, and thus can be considered “dumb phones” only. Symbian OS, the first smartphone system, began to support the installation of third-party apps and accounted for 67% market share in 2006. This system was used by many mobile phone brands, e.g., Nokia, Samsung, Motorola, Sony Ericsson, etc. Android and iOS, first released in 2007, signaled the start of the age of modern smartphones. Their app store ecosystems expanded quickly, with rich functionalities supported by the huge number of third-party apps. Therefore, they defeated other competitors, including Symbian, MeeGo, Windows Phone, etc., and dominate the smartphone market today. With so many third-party apps, libraries, and services, managing their access control is a nontrivial task.

Permission Model for Smartphones

Modern smartphones manage the access control of sensitive resources based on the permission model. In this model, the smartphone system defines one permission to access each sensitive resource, e.g., GPS, contact list, storage, photo access, etc. The smartphone system then enforces the mediation over the sensitive resources to ensure only the application with the corresponding permission can access the desired resource. Permissions requests should be explicitly declared by app developers, and permission approval is determined by smartphone users.

When application developers implement their app, they need to explicitly declare all the permission requests for their app. Developer documentation or manuals released by the smartphone systems can be referred to for the definition of permissions and when they should be requested. Then, the developers submit the application binary together with the permission declaration to the centralized app market for review. The centralized market vets the app binary for security violations, and signs the app if it passes the vetting. The signed app is returned to the app developers for signature, and then allowed to appear in the app market for user downloads. Since signatures cannot be forged–an app from other developers cannot claim to be from Microsoft, for instance–users can always choose apps from their trusted developers.

After users browse the market and decide to download the app package, the signature from the centralized market needs to be validated to ensure the app passed vetting and has maintained integrity after vetting. Then, the permission requests can either be approved during the installation time or runtime.

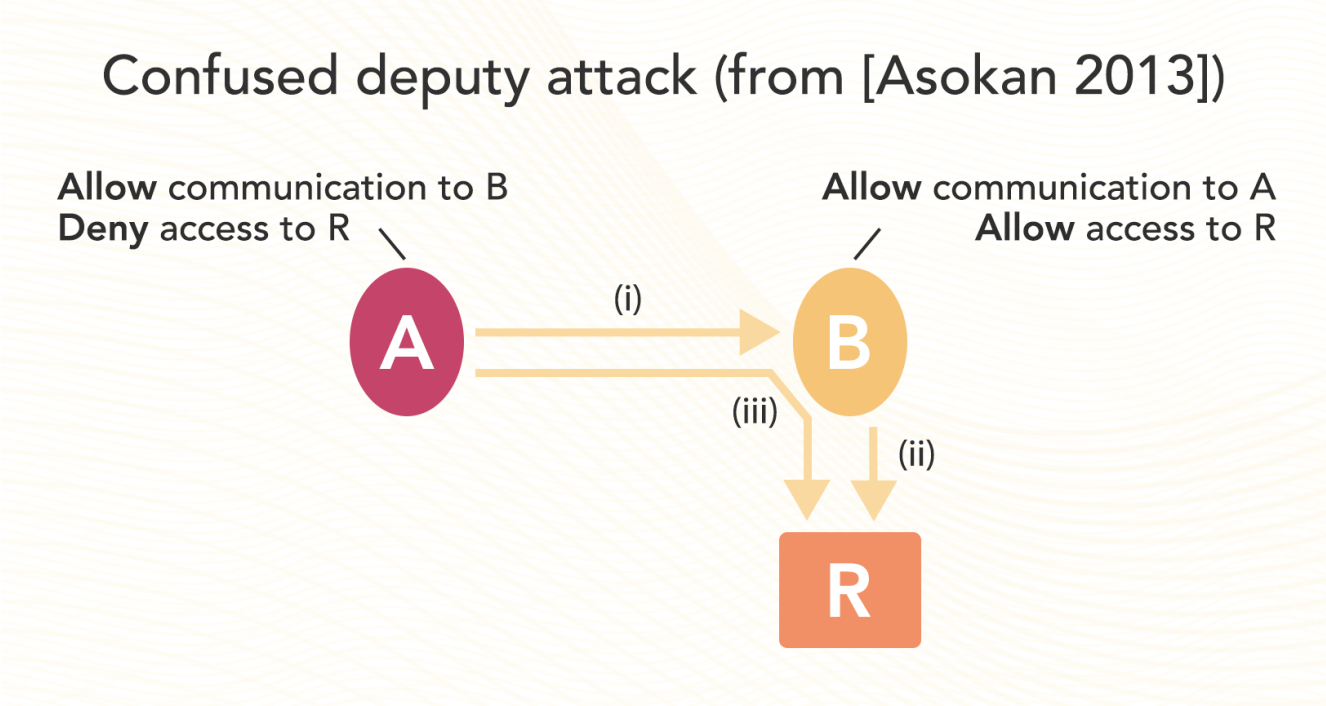
For the former, all the permission requests need to be presented to users for approval before installation. With users’ approval, the permissions are stored in a local database for the app, and the app can be installed on the smartphone. When users click the app to run it, the approved permissions will be loaded from the local database to the memory space of the app. If the app needs to access a sensitive resource mediated by the smartphone system during runtime, the system checks if the app has the corresponding permission approved or not. If so, the resource access will be allowed.

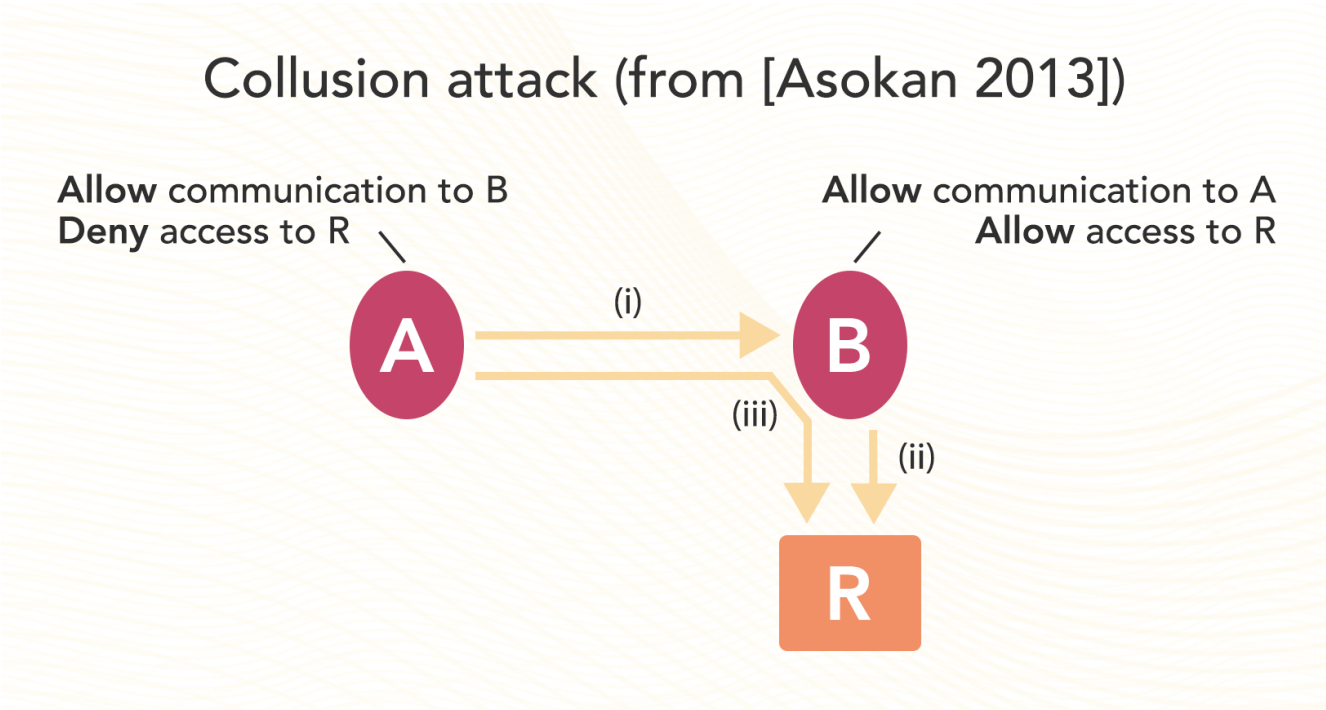
For the latter, the app is just installed on the smartphone after downloading. During its runtime, an attempt to access any sensitive resource will cause the smartphone system to present a dialog message to the user, asking the user to approve the permission request. Overall, for both installation time and runtime approval, users make the decision of whether the app should be given permission to access the sensitive resources on their smartphone.

Problems of the Permission Model

The permission model divides access control to sensitive resources on smartphones into permission request, permission approval, and permission enforcement, with developers, users, and smartphone systems responsible for each correspondingly. The logic is quite clear: each party should do what they are good at. For instance, developers should know every detail of their app, so they should know the exact permissions necessary for it to run correctly. Smartphone users should be aware of all the sensitive data on their phone, so they should have the full control of what each app can access on their phone. Smartphone systems should enforce access control to check for each attempted access to the sensitive resources during their design and implementation. The permission model approach well addresses the complicated resource authorization problem given the existence of the huge number of third-party apps, but it is still far from perfect.

Firstly, app developers tend to make more permission requests than necessary. The reason for requesting over-privilege can be twofold. On the one hand, developer documentation may not clarify the differences between similar permissions, so developers simply request all of them. On the other hand, developers may think other permissions will be necessary for their later updates, so they request them in advance. However, such violations of the least privilege principle can make the app suffer from a confused deputy attack, if it is induced by attackers to misuse its privileges. Secondly, users are not always knowledgeable enough to make correct access decisions, even when provided with detailed explanations of each permission. In reality, lots of users simply ignore permission requests (not to mention the detailed explanations), and quickly click approve without thinking if the requests make sense or not for the app they want to install. Thirdly, some permissions are defined too coarse-grained, also causing an over-privilege problem. For instance, a single system permission may be defined to include allowing volume control, preventing the phone from sleeping, retrieving a list of installed apps, etc. The app that needs to retrieve a list of installed apps has to request the system permission, and once approved, it is also allowed to prevent the phone from sleeping, even if that is not necessary for it. Finally, the permission model can limit the operations of one app, but cannot rule out the confused deputy attack or collusion attack. For instance, to protect a sensitive resource, R, one user is very conservative, not allowing any suspicious application (i.e., A) to access R by denying the corresponding permission request. However, as illustrated in the figure on the left below, A knows B is allowed to access R, and can launch a confused deputy attack to leverage B to access R on its behalf. Even worse, if B is also compromised, A and B can collude to access R. You probably want to ask, since B can access R directly and has been compromised, why is it necessary to collude with A? Consider if only A has permission to access the internet, and B does not. To leak R to a third-party server, A and B have to collude, so B accesses R and forwards it to A, which then leaks it out.





Some Solutions

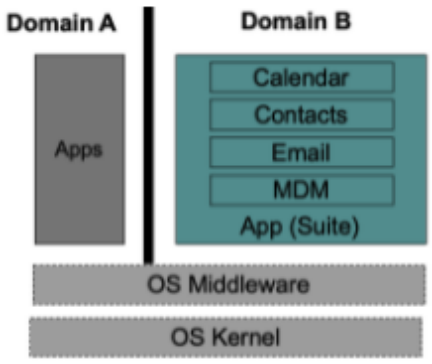
Static and Dynamic Analysis of Apps

Since third-party apps are not always trustworthy, analyzing their behavior is a straightforward solution to determine if they are legitimate or malicious. Generally, there are two approaches: static analysis and dynamic

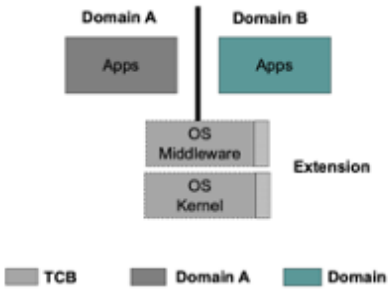
analysis. The former investigates the app binary or source code, if available, without actually running the app, while the latter monitors the app's behavior during its runtime. The advantage of static analysis is that it can examine all possible execution paths, since the entire code is investigated. Hence, some vulnerabilities that may not manifest themselves for weeks or months can be discovered. In contrast, dynamic analysis can identify vulnerabilities that depend on complicated inputs or events. One thing special about dynamic analysis of smartphone apps is the amount of interactive inputs required to activate different functionalities of the app. Some tools that can automate users' operations on the smartphone touch screen have been developed and widely used.

Isolation

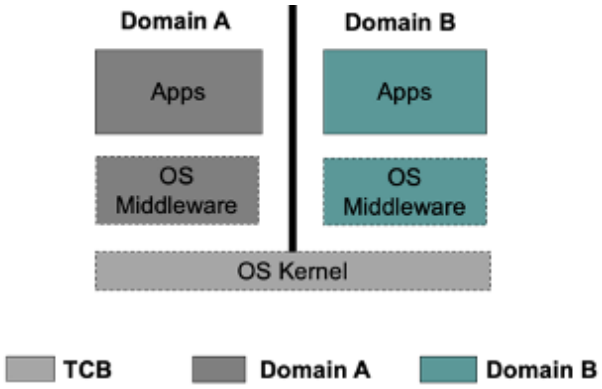
Separating smartphone apps into different domains is a common approach used by organizations to support BYOD (bring your own device). Enterprise apps are isolated in one domain, while all other apps are isolated in another domain to ensure minimal interference. Different mechanisms can be utilized to implement security domain isolation. For instance, Good for Enterprise and Enterproid Divide use an application-based approach to encrypt business data and applications and keep them separate from personal data. A MAC-based approach revises smartphone OS kernel and middleware (framework), thus enforcing the MAC policy to isolate two security domains, e.g., BizzTrust and Samsung KNOX.



Application Based Approach



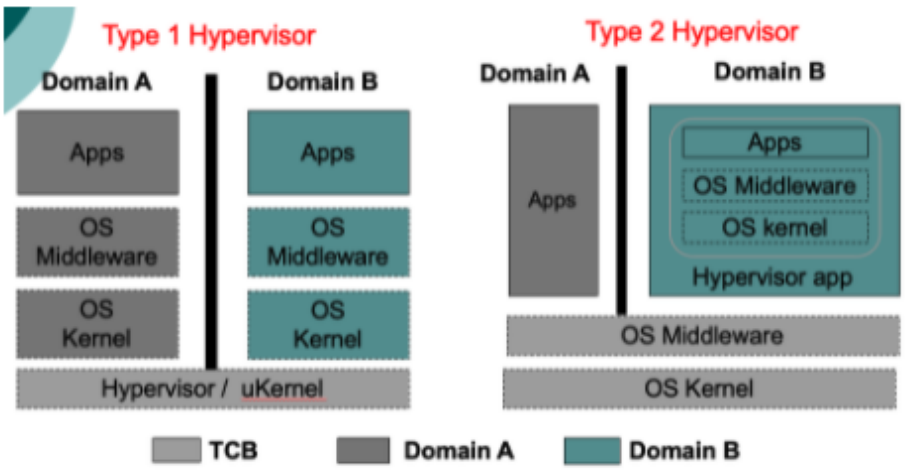
MAC based Approach



Kernel based Approach

A kernel-level approach utilizes the container technique to virtualize dedicated work spaces to run multiple smartphone OS middlewares and upper layer apps. The apps in different domains use their dedicated OS middleware to provide services without interference. Compared with the MAC-based approach whose TCB (trusted computing base) includes both OS kernel and middleware, the TCB only includes OS kernel here. Thus, even if the OS middleware in Domain A is compromised, the security of enterprise apps in Domain B will not be affected. This is not possible in a MAC-based approach, where the enterprise apps also rely on the correct execution of the single OS middleware. Cellrox is a kernel-level approach to isolate security domains.

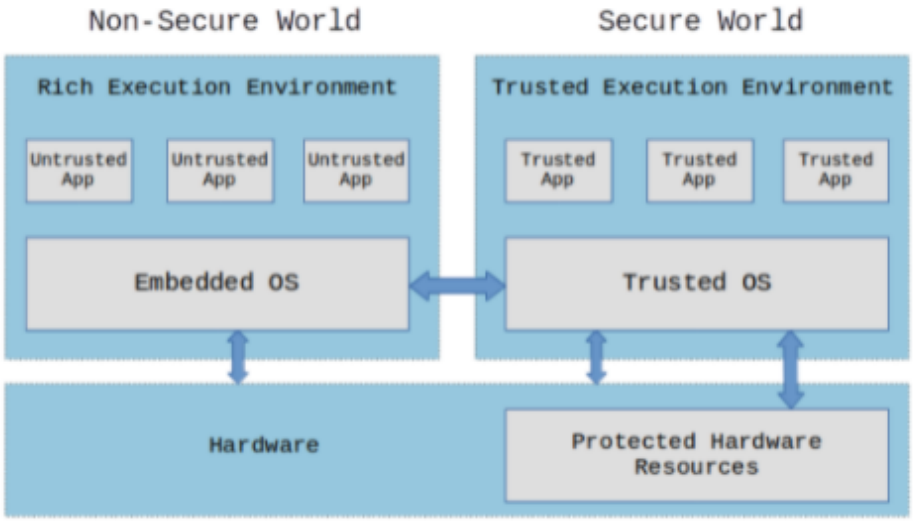
A virtualization-based approach relies on virtualization techniques to emulate VMs to isolate security domains. As we learned in our section on virtualization, two types of virtualization can be used: Type 1 hypervisor and Type 2 hypervisor. Enterprise apps can run inside a VM with a dedicated OS middle and OS kernel. Virtualization-based isolation is even stronger than kernel-level isolation, in which two OS middlewares in Domain A and Domain B still share the same OS kernel. VMware Mobile Virtualization Platform is one of such example of a virtualization-based approach.



Virtualization based approach

Finally, hardware-based isolation is also possible on ARM processors that support ARM TrustZone. In particular, a secure world can be isolated from a non-secure world based on protected hardware resources. The enterprise apps, considered trustworthy can run inside the secure world, while all other apps, considered untrustworthy, are

running in the non-secure world. Since the two worlds are isolated based on hardware, rather than software, the confidentiality of enterprise data and the integrity of the enterprise apps can be preserved.



Hardware-based approach

Topic 4: IoT Security

What Is IoT?

The Internet of Things (IoT) refers to a network that connects uniquely identifiable “things” to the internet. The “things” should have sensing or actuation and potential programmability capabilities, e.g., wireless motion sensors, surveillance cameras, smartphones, smartwatches, smart switches, smart lights, etc. The concept of IoT is pervasive these days, including, but not limited, to the following areas:

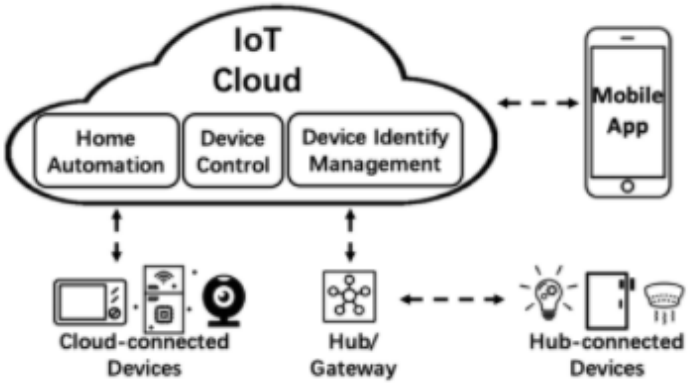
- Smart home - Home security systems equipped with surveillance cameras, motion sensors, etc., will report any intrusion or abnormality. Smart lights, switches, thermostats, and refrigerators work collaboratively to provide a comfortable living environment.
- Connected vehicles - Each vehicle collects traffic and surrounding information, and reports it back to the infrastructure to coordinate routes of local vehicles, facilitate autonomous driving, etc.
- Smart health - Medical devices can be implanted into the body of patients, actively monitoring their physical condition and alerting their doctor of any abnormality in real time.
- Smart manufacturing - Industry 4.0 advocates automation of traditional manufacturing and engineering practices, with the help of the widely integrated IoT technique to improve machine communication, self-monitoring, self-diagnosis, etc.
- Smart grid - Smart meters communicate real-time power consumption information to electricity suppliers for system monitoring. Efficient energy distribution and load balancing are based on real-time data collected by smart sensors everywhere.

Architecture of IoT

Take smart home as an example. IoT typically includes three major components: the IoT cloud, IoT devices, and mobile apps. Firstly, the IoT cloud is the management platform for smart home devices, with three major functions: device identification management, device control, and home automation. Device identification maintains the binding between the IoT devices and their owner's account, device control serves as a proxy forwarding users' remote commands to the corresponding devices, and home automation allows users to define customized rules to control devices automatically. Secondly, IoT devices are equipped with sensors to collect surrounding information or actuators to perform operations, e.g., smart switches, smart lights, smart cameras, etc. There are generally two ways for them to connect to the internet, via WiFi to the home router or via Z-Wave/Zigbee to a smart hub. Lastly, mobile apps allow users to interact with the IoT devices indirectly, including for initial device setup, remote control, rules customization, etc., since most of those IoT devices do not have I/O interfaces (e.g., a screen, keyboard, etc.). The architecture of IoT used in other areas is more or less the same as that in smart homes.

The lifecycle of an IoT device can be summarized as below:

- Device discovery - To setup a new IoT device, the user downloads the companion mobile app, registers an account, and clicks the "add a new device" button to identify a nearby IoT device. After establishing a connection with the smartphone, the device reports its basic information, e.g., MAC address, device model, etc.
- WiFi provisioning - The user configures the WiFi connection credentials on the mobile app to connect the IoT device to the local network.
- Device registration and binding - A unique device ID needs to be registered with the IoT cloud platform. Afterwards, the IoT cloud binds the device ID with the user's account, so only the authorized user can access the device through the IoT cloud.
- Device login - The IoT device uses its ID to log into the IoT cloud to update its status and receive the user's commands to operate.
- Device unbinding and reset - When the user decides to reset or abandon the device, the user can unbind the device and make it return to the factory mode.



Smart Home Platform Architecture (from [Zhou 2009])

Security Problems in IoT

Since IoT consists of three major entities as introduced above, the security issues of each entity contribute to the entire IoT system attack surface. For instance, the confidentiality issue of cloud computing makes the IoT cloud suffer from leakage of users' sensitive data, e.g., home address, calendar information, location of surveillance cameras, etc. The integrity issue causes the IoT cloud to report incorrect information to users, thus inducing users to send wrong control commands, e.g., revising low temperature to be high causes users to open smart windows. Similarly, compromising the companion app or smartphone platform gives the attackers full control over the IoT devices as well. Finally, since most of the IoT devices are not expensive and can be easily purchased, attackers can physically access and analyze them. IoT devices of the same type from the same manufacturer typically share the same configuration, parameters, and default passwords. For instance, on October 21, 2016, attackers leveraged the Mirai malware to control lots of IoT devices as part of a botnet, and launched a DDoS attack against DNS services causing inaccessibility of several high-profile websites. (For more detail, see Today the web was broken by countless hacked devices – your 60-second summary.)

The breakout of Mirai malware mainly results from the use of default username and password on lots of IoT devices. Some manufactures still keep the debug port open in their released products (as was the case with the first version of Amazon Echo), which allows attackers to retrieve the firmware running inside the devices. Some may even have the same cryptographic key hard-coded in the firmware running inside the devices.

Another source of threats comes from the interaction among the three entities. Lack of synchronization, unauthorized device login, unauthorized device unbinding, and other issues have been reported (Zhou, 2019). Meanwhile, vulnerability analysis of the protocols used in IoT (e.g., REST, MQTT, OAuth, etc.) also has revealed various vulnerabilities.

Test Yourself

Users can always make the correct decision to approve permission requests from apps.

True

False

Test Yourself

Describe the confused deputy and collusion attacks in smartphone systems.

Refer to the Permission Model of Smartphone section in the lecture notes.

Test Yourself

Compare the TCB of different isolation approaches and state your conclusion.

The TCBs of an application-based approach and a MAC-based approach include the OS middleware and OS kernel. The TCB of kernel-level approach includes the OS kernel only. The TCB of a Type 1 hypervisor includes the hypervisor only, while the TCB of a Type 2 hypervisor includes the Host OS kernel and Host OS middleware. As the isolation approach goes to the lower layer, the size of the TCB becomes smaller. Typically, a smaller TCB can ensure its code base can be automatically verified for correctness, thus keeping it error- or vulnerability-free.

Test Yourself

Briefly describe the permission model used by the smartphone system you use.

Answers may vary; refer to the Problems of Permission section in the lecture notes.

Test Yourself

Using the default username and password is a common problem of IoT devices.

True

False

Test Yourself

Which one of the following is not a smartphone system?

Java ME

Symbian

Android

iOS

Window Phone OS

Test Yourself

Describe the roles of mobile apps and the cloud in the IoT architecture.

Refer to the IoT Architecture section in the lecture notes.

References

- • Pfleeger C. P., Pfleeger, S. L., & Marguiles, J. (2015). *Security in computing* (5th ed.) Prentice Hall.
- Du, W. (2019). *Computer & internet security: A hands-on approach* (2nd ed.).(n.p).
- Garfinkel, T., & Rosenblum, M. (2003). *A virtual machine introspection based architecture for intrusion detection.* NDSS Symposium 2003.
- King, S. T., Chen, P. M., Wang, Y., Verbowski, C., Wang, H., & Lorch, J. (2006). *SubVirt: Implementing malware with virtual machines. Proceedings of the 2006 IEEE Symposium on Security and Privacy.*
- NVISH Solutions Inc. (2011). *Roadmap to cloud computing.*
- Zhou, W., Jia, Y., Yao, Y., Zhu, L., Guan, L., Mao, Y., Liu, P., & Zhang, Y. (2019). *Discovering and understanding the uecurity hazards in the interactions between IoT Devices, mobile apps, and clouds on smart home platforms.* USENIX Security Symposium, 2019.
- Asokan, N., Davi, L., Dmitrienko, A., & Heuser, S. (2013). *Mobile platform security.* Morgan & Claypool.